

HW 04 — White Dwarf Structure via Multi-Dimensional Newton's Method

Overview

This assignment builds a multidimensional Newton solver and applies it to compute the hydrostatic structure of a white dwarf star. The star is modeled as n concentric mass shells with a polytropic equation of state; the unknowns are the shell-edge radii $\{r_i\}_{i=1}^n$.

Part 1: Multi-Dimensional Newton Solver — `newton_nd`

Problem Statement

Implement `newton_nd(F, J, x0, ...)` that solves the vector equation $F(\mathbf{x}) = \mathbf{0}$ using Newton's method. $F: \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the residual vector; $J = \partial F / \partial \mathbf{x}$ is the Jacobian matrix. An optional `accept_step` callback can damp the Newton step to keep the solution in a physically valid region.

Algorithm

At each iteration, Newton's method linearizes F around the current guess $\mathbf{x}^{(k)}$:

$$J(\mathbf{x}^{(k)}) \Delta \mathbf{x} = -F(\mathbf{x}^{(k)})$$

Solve this linear system for $\Delta \mathbf{x}$, then update $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha \Delta \mathbf{x}$, where $\alpha \in (0, 1]$ is the damping factor from `accept_step`.

Convergence criteria: Stop when $\|F(\mathbf{x})\|_\infty < \varepsilon_F$ (residual small) or $\|\alpha \Delta \mathbf{x}\|_\infty < \varepsilon_x \|\mathbf{x}\|_\infty$ (step small relative to current value).

Quadratic convergence: Near the root, Newton's method satisfies $\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \lesssim C \|\mathbf{x}^{(k)} - \mathbf{x}^*\|^2$. In the test case (exact solution $(1, 2)$), the residual history $[0.96, 0.049, 4.7 \times 10^{-4}, 6.9 \times 10^{-8}, 1.8 \times 10^{-15}]$ shows the characteristic quadratic doubling of significant digits.

Implementation:

```
for iteration in range(max_iter):
    fx = np.asarray(F(x), dtype=float)
    if np.linalg.norm(fx, ord=np.inf) < tol_f:
        return x, {"converged": True, ...}
    jac = np.asarray(J(x), dtype=float)
    dx = np.linalg.solve(jac, -fx)      # solve J Δx = -F
    factor = 1.0 if accept_step is None else float(accept_step(x, dx))
    x = x + factor * dx
```

`np.linalg.solve` uses LU decomposition internally — $O(n^3)$ per iteration.

Part 2: The White Dwarf Model

Physical Setup

A white dwarf is supported against gravity by **electron degeneracy pressure**, modeled as a polytropic equation of state:

$$p = \kappa \rho^\gamma$$

with $\gamma = 5/3$ (non-relativistic) or $\gamma = 4/3$ (ultra-relativistic).

The star is divided into n mass shells. The mass grid is **fixed** and uniform:

$$m_i = i \cdot M_{\text{total}}/n, \quad i = 0, 1, \dots, n$$

The shell-edge radii $\{r_1, \dots, r_n\}$ are the **unknowns**. From the radii, density and pressure in each shell follow:

$$\rho_i = \frac{\Delta m_i}{(4\pi/3)(r_i^3 - r_{i-1}^3)}, \quad p_i = \kappa \rho_i^\gamma$$

The Residual Equations

Hydrostatic equilibrium (pressure gradient balances gravity) in finite-difference form gives one equation per shell interface:

$$f_i = \frac{p_{i+1} - p_i}{\Delta m_i^*} + \frac{G m_i}{4\pi r_i^4} = 0, \quad i = 1, \dots, n$$

This is a **forward** difference of the hydrostatic equation $dp/dm = -Gm/(4\pi r^4)$. Here $\Delta m_i^* = \frac{1}{2}(m_{i+1} - m_{i-1}) = M/n$ is the mass associated with vertex i , and the pressure just outside the outermost shell is set to zero, $p_{n+1} \equiv 0$ (surface boundary condition: zero pressure at the star's edge).

The second term is the gravitational acceleration term. The code also uses a **scaled residual** (dividing f_i by the gravity term) so all components have magnitude ~ 1 near the solution, which improves Newton convergence.

The Analytic Jacobian

$\partial f_i / \partial r_j$ is derived analytically by differentiating the density and pressure formulas. Since $p_k = \kappa \rho_k^\gamma$, the chain rule gives:

$$\frac{\partial p_k}{\partial r_j} = \gamma \frac{p_k}{\rho_k} \cdot \frac{\partial \rho_k}{\partial r_j}, \quad \frac{\partial \rho_k}{\partial r_k} = -\frac{3\rho_k r_k^2}{r_k^3 - r_{k-1}^3}, \quad \frac{\partial \rho_k}{\partial r_{k-1}} = +\frac{3\rho_k r_{k-1}^2}{r_k^3 - r_{k-1}^3}$$

which yields the pressure derivatives actually assembled in the code:

$$\frac{\partial p_k}{\partial r_k} = -\frac{3\gamma p_k r_k^2}{r_k^3 - r_{k-1}^3}, \quad \frac{\partial p_k}{\partial r_{k-1}} = +\frac{3\gamma p_k r_{k-1}^2}{r_k^3 - r_{k-1}^3}$$

Each pressure p_k depends only on the two radii bounding its shell (r_{k-1} and r_k), and each residual f_i involves p_i and p_{i+1} ; therefore f_i depends on r_{i-1}, r_i, r_{i+1} and J is **tridiagonal**. The analytic Jacobian is verified against a finite-difference approximation; relative error $\approx 5 \times 10^{-11}$ confirms correctness.

Using the analytic Jacobian instead of a numerical one saves n extra residual evaluations per Newton step and avoids finite-difference truncation error.

Damping: Monotonic Radius Step

During early Newton iterations, an unconstrained step might make some radii decrease past each other (making shell volumes negative — unphysical). The `accept_step` callback `monotonic_radius_step_factor` finds the largest $\alpha \leq 1$ such that all shell widths remain positive after taking the step $r + \alpha \Delta r$. This keeps the solution in the feasible domain.

Results

Non-relativistic ($\gamma = 5/3$): The well-known analytical result is $R \propto M^{-1/3}$, or equivalently $M^{1/3}R = \text{const}$. The scan over $M = 0.4, 0.6, 0.8, 1.0, 1.2 M_\odot$ yields $M^{1/3}R \approx 0.01220$ for all masses, confirming the $M^{-1/3}$ scaling to 6 significant figures.

For $M = 1.0 M_\odot$: $R \approx 0.0122 R_\odot$, which is a realistic white dwarf radius.

Near-relativistic ($\gamma = 1.334$, approaching $4/3$): As $\gamma \rightarrow 4/3$, the polytropic model predicts a limiting Chandrasekhar mass $M_{\text{Ch}} \approx 1.44 M_\odot$ above which no equilibrium exists. The simulation shows the radius shrinking precipitously as $M \rightarrow M_{\text{Ch}}$: from $R \approx 557 R_\odot$ at $M = 1.41$ to $R \approx 0.05 R_\odot$ at $M = 1.45$, and non-convergence at $M = 1.46$. This sensitivity near the Chandrasekhar limit is a genuine physical effect captured by the model.

SciPy comparison: `scipy.optimize.root` with the `hybr` method (a variant of Powell’s hybrid method) gives the same radii to within the tolerances set, confirming the Newton implementation is correct.

Key Takeaways

- **Newton’s method generalizes to systems:** at each step solve the linear system $J \Delta \mathbf{x} = -F$ and update $\mathbf{x} \rightarrow \mathbf{x} + \Delta \mathbf{x}$. The Jacobian $J = \partial F / \partial \mathbf{x}$ replaces the scalar derivative.
- **Quadratic convergence** near the root — the number of correct digits roughly doubles each iteration — but only if the initial guess is good enough.
- **Prefer an analytic Jacobian** when you can derive it: it avoids finite-difference truncation error and saves n residual evaluations per step. Always verify it against a finite-difference check.
- **Exploit sparsity.** Here each residual couples only neighboring shells, so J is **tridiagonal** — cheap to solve. Recognizing structure is what makes large systems tractable.
- **Damping/globalization keeps Newton physical.** An undamped step can leave the valid domain (negative shell volumes); a step-limiting factor $\alpha \in (0, 1]$ keeps the iterate feasible.
- **Scale the residual** so all components are $O(1)$ near the solution — this conditions the linear system and improves convergence.
- **Physics insight:** the polytrope reproduces the white-dwarf mass-radius law $R \propto M^{-1/3}$ for $\gamma = 5/3$, and the Chandrasekhar instability (radius collapsing, solver failing to converge) as $\gamma \rightarrow 4/3$ — the numerics mirror the physics.